Path Planning to Avoid Obstacles and Partially Blocked Roads

Team 8

Guangzhou Cai (guangzhc@andrew.cmu.edu)
Ting-You Chen (tingyouc@andrew.cmu.edu)
Ziang Jiao (ziangj@andrew.cmu.edu)

Project Repo: https://github.com/JiaoZA01/viplanner

Project Abstract

This project develops a hybrid navigation system that improves the reliability of learning-based motion planners in complex environments. While ViPlanner can generate smooth trajectories using visual and semantic inputs, we observed that it frequently fails in partially blocked or occluded scenarios due to limited field-of-view and lack of persistent memory.

To address these issues, we integrate a D* Lite–based planner as a safety and fallback mechanism. The system constructs a local occupancy grid from depth observations and uses D* Lite to verify whether the trajectory proposed by ViPlanner is collision-free. If the learned planner produces an unsafe or infeasible path, the system switches to a graph-based path generated by D* Lite. Additionally, we implement a persistent world-frame obstacle memory to improve robustness in scenarios where obstacles move out of view.

The system is implemented in NVIDIA Isaac Sim using a four-wheel robot platform. Through scenario-based testing, we show that the hybrid approach significantly improves success rate and reduces collision cases in environments with dead-ends, narrow passages, and occlusions, while preserving the efficiency of the learned planner in standard conditions.

Introduction

Autonomous navigation systems must balance two competing goals: adaptability to complex environments and reliable safety guarantees. In practice, purely learning-based planners such as ViPlanner can generate smooth and efficient trajectories from visual and semantic inputs, but we observed consistent failure cases when deploying them in simulation. In particular, the planner struggles in environments with limited field-of-view, partial occlusions, and dead-end structures, often committing to unsafe paths or getting trapped in local minima.

These issues arise from two key limitations. First, the planner operates primarily on local observations and lacks persistent memory of previously seen obstacles, which leads to short-sighted decisions. Second, the learned cost representation does not provide explicit safety guarantees, meaning the system may still output trajectories that are physically infeasible or lead to collisions.

To address these limitations, we design a hybrid navigation framework that augments the learned planner with a classical D* Lite–based planning module. Instead of replacing the neural planner, our system introduces a lightweight checker that validates proposed trajectories against an occupancy grid constructed from depth observations. When the learned trajectory is unsafe or infeasible, the system switches to a deterministic fallback path generated by D* Lite. We further extend this design with a persistent obstacle memory to improve robustness in partially observable environments.

Through this integration, the system combines the efficiency and smoothness of learning-based planning with the reliability of classical graph search, resulting in a more robust navigation pipeline for challenging real-world scenarios.

Project Motivation and Overview

During initial testing of ViPlanner in simulation, we observed that while the planner performs well in open and structured environments, it consistently fails in more constrained scenarios such as partially blocked roads, narrow passages, and occluded turns. In these cases, the robot often commits to trajectories that appear locally valid but lead to dead-ends or collisions.

Two recurring failure patterns motivated this work. First, due to its limited field-of-view and lack of memory, the planner makes short-sighted decisions and can become trapped in local minima (e.g., U-shaped obstacles or blocked corridors). Second, the learned cost representation does not explicitly enforce safety constraints, which allows the system to occasionally generate trajectories that pass too close to obstacles or even through invalid regions.

These observations highlight a gap between the flexibility of learning-based planners and the reliability required for safe navigation. Rather than replacing the learned planner, we aim to augment it with a lightweight, deterministic mechanism that can detect unsafe behavior and provide a safe fallback when necessary.

Requirement and Use Cases

Requirements

Trajectory Generation

- The system shall generate a feasible navigation trajectory from the robot's current state to a target goal using a learning-based planner (ViPlanner).

Safety Validation (Checker)

- The system shall validate the proposed trajectory against an occupancy grid derived from depth observations.
- The system shall reject trajectories that intersect or come within a safety margin of obstacles.

Fallback Planning

- The system shall generate an alternative collision-free path using D* Lite when the primary trajectory is unsafe or infeasible.

Planner Switching

- The system shall dynamically switch between ViPlanner and D* Lite based on safety validation results.

Occupancy Grid Construction

- The system shall convert depth observations into a local occupancy grid representation for path validation and planning.

Persistent Obstacle Memory

- The system shall maintain a world-frame memory of previously observed obstacles to support planning beyond the current field-of-view.

Path Execution

- The system shall convert selected trajectories into velocity commands and execute them on a differential-drive robot.

Use Case

The following use cases illustrate how the proposed hybrid navigation system operates under challenging conditions where purely learning-based planners typically fail. Each scenario highlights the limitations of the baseline approach and how the integration of the checker and D* Lite planner improves system robustness and safety:

1. Dead-End and Local Minimum Scenario

In environments with U-shaped obstacles or partially blocked corridors, the robot may encounter situations where the goal lies ahead but no feasible path exists. A purely reactive planner such as ViPlanner tends to commit to forward motion based on local observations and may become trapped or collide with obstacles.

In the proposed system, the checker evaluates the planned trajectory against the occupancy grid and detects that the path leads into blocked space. The system then switches to the D* Lite planner, which identifies a valid path to exit the dead-end and guides the robot toward a feasible route.

2. Occluded or Blind Turn Scenario

When navigating around corners or through occluded regions, the robot's field-of-view is limited, and obstacles may not be immediately visible. In such cases, a learning-based planner may generate trajectories that assume free space, leading to unsafe behavior.

The hybrid system addresses this by validating the trajectory using the occupancy grid and, when necessary, incorporating persistent obstacle memory. If the proposed path is deemed unsafe, the system slows down and switches to a D* Lite-generated path, allowing the robot to safely navigate around the occlusion.

3. Narrow Passage and Clearance Constraint Scenario

In tight environments where the robot must pass through narrow gaps, precise control and sufficient clearance from obstacles are required. ViPlanner may generate trajectories that are too close to obstacles or violate safety margins.

To handle this, the system applies obstacle inflation within the occupancy grid to enforce a safety buffer. The checker rejects trajectories that violate this constraint, and the D* Lite planner generates a safer, centered path that maintains appropriate clearance while navigating through the passage.

4. Sensor Noise and Perception Error Scenario

In the presence of sensor noise or imperfect perception (e.g., missing or misclassified obstacles), the learning-based planner may produce invalid trajectories that intersect with obstacles.

The checker module mitigates this issue by validating the trajectory against the occupancy grid constructed from depth observations. If inconsistencies are detected, the system

rejects the unsafe trajectory and either switches to a D* Lite path or executes a conservative fallback behavior, such as slowing down or reorienting.

5. ***Out-of-View Obstacle and Memory Scenario***

As the robot moves, previously observed obstacles may leave the current camera view but still pose a risk. A purely reactive planner may fail to account for these obstacles and generate unsafe trajectories.

The proposed system maintains a persistent world-frame memory of observed obstacles. This memory is used to reconstruct the occupancy grid when needed, enabling D* Lite to plan around obstacles that are no longer visible. As a result, the robot can maintain safe navigation even in partially observable environments.

System Architecture and Description (System Design & Block Diagram)

The system design, as illustrated in the provided architecture, separates "Intuition" (Neural Network) from "Logic" (Deterministic Algorithm) to create a safe, hybrid navigation stack.

The Dual-Path Pipeline

The architecture begins with the Data Acquisition layer (Depth and Semantic data). This data feeds into two parallel streams:

The Neural Stream (Viplanner):
Data enters the Perception Networks to generate latent embeddings. These are processed by the Planning Networks, which output a suggested neural trajectory (Trajectory v).

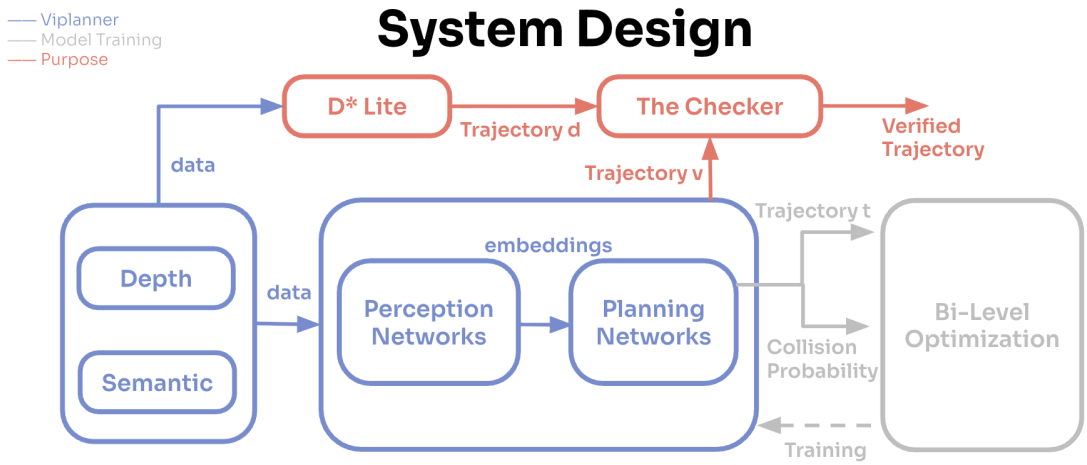
The Symbolic Stream (D* Lite):
Simultaneously, raw data is sent to the D* Lite algorithm. This module maintains an incremental graph and calculates a mathematically optimal symbolic trajectory (Trajectory d).

The Checker and Bi-Level Optimization

The Checker module acts as the final arbitrator of the system. It evaluates Trajectory v against the constraints established by Trajectory d . If the neural prediction is found to be

unsafe, or if it has entered a local minimum (such as a dead-end), the Checker overrides the neural output with the verified symbolic path to produce the final Verified Trajectory.

The Bi-Level Optimization block serves as the mathematical foundation for the Checker's decision-making process. Instead of being used for model training, this optimization layer calculates the Collision Probability and refines the trajectories in real-time. It ensures that the final control signal is a result of a bi-level objective: minimizing the distance to the goal (global optimality) while maintaining a zero-collision threshold (local safety). This dual-layer validation provides a deterministic "safety architecture" that bridges the gap between AI intuition and formal safety requirements.



System Implementation including API and their brief description (quantitative evaluation)

1. System Overview

The implemented system uses a hybrid autonomous navigation architecture that combines a learning-based local planner, ViPlanner, with a classical D* Lite safety planner. ViPlanner provides fast local navigation from visual input, while D* Lite provides an explicit graph-search-based path validation and replanning mechanism. The purpose of this design is not to replace the neural planner, but to add a deterministic checking layer that can detect unsafe or blocked trajectories and switch the robot toward a safer fallback path when necessary.

ViPlanner is used as the main semantic local planner because it is designed to generate local plans from depth and semantic perception, using geometric and semantic information rather than only obstacle geometry. The public ViPlanner repository describes it as a learning-based local path planner using semantic and depth images, implemented as an Isaac Sim / Isaac Lab extension and also available for ROS integration. (GitHub) The corresponding paper describes ViPlanner as a visual semantic local navigation method trained in simulation with imperative learning, using a differentiable semantic costmap to reason about traversability. (arXiv)

In our implementation, the high-level navigation loop is:

1. Receive depth, semantic, and camera transform observations from the Isaac Lab environment.
2. Generate or receive a candidate path from the planner.
3. Convert depth perception into a local occupancy grid.
4. Use D* Lite to compute a conservative collision-aware path through the grid.
5. Use the checker to validate whether the current path is safe.
6. If the path is valid, continue using the nominal ViPlanner path.
7. If the path is unsafe or blocked, switch to the D* Lite fallback path.
8. Convert the selected path into velocity commands.
9. Convert velocity commands into differential-drive wheel commands for the robot.

This structure gives the system two complementary properties: the flexibility of a learned visual planner and the safety of an explicit grid-based search planner.

2. Perception and Observation API

The environment configuration defines several observation groups for policy execution, image-based planning, and camera pose tracking. The policy observation group includes robot state information such as base linear velocity, base angular velocity, projected gravity, velocity command, joint positions, joint velocities, previous actions, and height scan data. The planner image group provides depth and semantic camera measurements, while the planner transform group provides the camera position and orientation needed to transform obstacle observations between camera and world coordinates.

The key perception API can be summarized as:

```python
planner_image.depth_measurement
planner_image.semantic_measurement
planner_transform.cam_position
```

```
planner_transform.cam_orientation
```

The depth image is used directly by the D* Lite module to construct a local obstacle grid. The semantic image remains part of the ViPlanner-side perception pipeline, where semantic information can help the learned planner distinguish between traversable and non-traversable regions. The camera position and orientation are especially important for maintaining persistent obstacle memory across frames, because local depth observations must be transformed into a consistent world-frame representation before they can be reused.

3. Occupancy Grid Construction

The D* Lite module constructs a local occupancy grid from the depth camera. In our implementation, get_d_star_path() creates a GridWorld with a fixed local grid size of 80 x 80 cells and a cell resolution of 0.1 m. The robot starts at the bottom-center of the grid, while the goal is projected into the grid using the local camera-frame goal coordinates.

Depth pixels are sampled at a fixed stride to reduce computational cost. Each valid depth pixel is back-projected into a 3D camera-frame point using the camera intrinsics:

Python
```
x = (u - cx) * z / fx
y = (v - cy) * z / fy
```

Points with invalid depth, excessive depth, or ground-like vertical position are filtered out. This prevents the floor from being incorrectly treated as an obstacle. The implementation also filters out points near the goal cube so that the target itself is not mistakenly inserted as an obstacle. After filtering, each obstacle point is mapped into a grid cell and inflated by a circular safety margin. This inflation step makes the planner more conservative by expanding occupied regions around observed obstacles.

The occupancy grid uses:

None
```
0   = free cell
-1  = obstacle or inflated obstacle
```

This grid becomes the input graph for D* Lite.

4. Grid Graph Representation

The D* Lite planner operates on a graph abstraction of the occupancy grid. Each grid cell is represented as a node with a string ID such as x15y0. The GridWorld class generates this graph automatically from the grid dimensions. By default, the graph supports 8-connected motion, meaning the robot can move horizontally, vertically, or diagonally between neighboring cells. Straight moves have cost 1.0, while diagonal moves have cost sqrt(2).

This graph representation is important because it allows D* Lite to reason over discrete traversability. Instead of directly controlling continuous robot motion, D* Lite first finds a sequence of safe grid states. These states are then converted back into local metric waypoints of the form:

```
Python
        [forward_x, horizontal_y, height_z]
```

The implementation explicitly maps grid coordinates back into the robot frame, where grid y corresponds to forward motion and grid x corresponds to lateral offset.

5. D Lite Path Planner*

The D* Lite planner is used as the deterministic safety planner. The implementation imports and uses the following planner API:

```
Python
        initDStarLite()
        scanForObstacles()
        computeShortestPath()
        nextInShortestPath()
```

The planner initializes the search from the robot's current grid cell to the projected local goal. It then scans the grid for obstacles, updates affected graph edges, and computes the shortest safe path. Occupied cells are represented by negative grid values, and when an obstacle is detected, edges connected to that obstacle cell are assigned infinite cost so that the path planner will avoid them.

The main D* Lite planning sequence is:

```python
queue = []
k_m = 0
graph, queue, k_m = initDStarLite(graph, queue, s_start, s_goal, k_m)
scanForObstacles(graph, queue, s_start, 100, k_m)
computeShortestPath(graph, queue, s_start, k_m)
path_points = _extract_path(graph, s_start, s_goal, X_DIM, CELL_RES)
```

After the shortest path is computed, _extract_path() repeatedly calls nextInShortestPath() to recover the next best node along the path. Each selected grid node is converted into a continuous waypoint for downstream path following.

6. Persistent World-Frame Obstacle Memory

A limitation of a pure local depth grid is that obstacles can disappear from the camera view as the robot moves, even though they still matter for planning. To reduce this issue, the implementation includes a persistent world-frame obstacle memory. Observed obstacle cells are transformed from camera frame into world coordinates and stored in _world_memory, using a resolution matched to the local grid resolution.

The memory API includes:

```python
_world_memory = {}
_MEM_RES = 0.1

clear_memory()
print_world_memory_map()
_build_grid_from_memory()
```

When the current local view cannot produce a safe path, the system falls back to a memory-based grid. This grid is rebuilt by projecting remembered world-frame obstacle cells back into the current camera frame. If the planner fails on the immediate depth grid, it attempts to replan using this stored map.

This improves robustness because the robot is no longer limited to only the obstacles visible in the current frame.

7. Checker and Switching Logic

The checker layer is responsible for deciding whether the nominal path should be trusted or whether the system should switch to the D* Lite path. The core safety check is

implemented through _path_has_clearance(), which verifies that candidate path waypoints are inside the grid and not too close to occupied cells. A path is rejected if any checked waypoint falls outside the known grid or intersects an obstacle neighborhood.

Conceptually, the checker performs the following validation:

```
None
        Input: candidate path
        For each waypoint:
            Convert waypoint to grid cell
            Check if the cell is inside the occupancy grid
            Check whether the cell is near an obstacle
        If all checks pass:
            accept path
        Else:
            reject path and trigger fallback
```

In the current system, the D* Lite path itself is also checked after extraction. If the D* Lite path has insufficient clearance, the path is discarded and the system either replans using memory or generates a turning fallback behavior.

This checker is the main connection point between the learning-based and classical components. ViPlanner remains the nominal planner, but the checker prevents blind execution of a path that violates the local occupancy constraints.

8. Fallback Behavior

The system includes multiple fallback levels:

First, D* Lite tries to find a path using the current depth-derived occupancy grid. If this fails, the planner attempts to reconstruct a grid from persistent world-frame obstacle memory and reruns D* Lite. If memory-based planning also fails, the system generates a simple turn path toward the side of the goal.

The final fallback behavior is:

```
Python
        turn_dir = 1.0 if goal_cam[1].item() >= 0 else -1.0
        turn_path = [[CELL_RES, turn_dir * i * CELL_RES, 0.0] for i in
        range(1, 6)]
```

This is not a full navigation solution, but it prevents the robot from blindly driving forward into blocked space. Instead, it causes the robot to rotate or bias motion toward the goal side until a better path becomes visible.

9. Path Action Interface

The selected path is passed into the action system through NavigationAction. The action interface stores the incoming path as a flat action vector and reshapes it into a tensor of shape:

```python
(num_envs, path_length, 3)
```

The configuration sets path_length = 51, meaning each path action contains 51 local or world-frame waypoints, each with 3 coordinates.

The relevant API is:

```python
process_actions(actions)
apply_actions()
```

process_actions() stores the raw path and reshapes it into the processed path tensor. apply_actions() then converts the path into low-level velocity commands. This design separates path generation from robot actuation: the planner only needs to output a waypoint path, while the action module handles tracking and motor command generation.

10. Path Following and Actuation

The actuation layer converts the selected path into robot velocity commands. The path follower first identifies a target waypoint ahead of the robot using a lookahead rule. It finds the closest point on the path, then selects a future point at least 0.8 m away when possible. This target is transformed from the world frame into the robot frame using the inverse robot orientation.

The controller then computes:

```python
x_err = target_vec_b[:, 0]
y_err = target_vec_b[:, 1]
```

```
angle_error = torch.atan2(y_err, x_err)
```

A turn-then-drive control policy is used. Linear velocity is proportional to forward error, while angular velocity is proportional to heading error. The implementation clamps the forward velocity and yaw rate to avoid unstable commands:

```python
v_raw = torch.clamp(1.2 * x_err, 0.0, 1.0)
w_raw = torch.clamp(5 * angle_error, -1.5, 1.5)
v_scale = torch.clamp(1.0 - torch.abs(angle_error) / 0.6, 0.0, 1.0)
v_cmd = v_raw * v_scale
omega_cmd = w_raw
```

The important behavior is that the robot slows down when its heading error is large. This prevents the robot from driving forward aggressively while it is still pointed away from the selected path. If the robot is within 0.3 m of the final goal, both linear and angular velocity are set to zero.

For the Limo differential-drive robot, the final linear and angular velocity commands are converted into left and right wheel angular velocities:

```python
omega_left  = (v - w * track_width / 2.0) / wheel_radius
omega_right = (v + w * track_width / 2.0) / wheel_radius
```

These wheel velocities are then applied directly to the robot joint velocity targets.

11. Environment Configuration

The Isaac Lab environment is configured as a manager-based reinforcement learning environment, but for this project it is used primarily as a simulation and control framework rather than a training pipeline. The action configuration disables the original ANYmal low-level policy by setting low_level_policy_file=None, and instead uses a custom LimoDiffDriveActionCfg with the Limo wheel joint names, wheel radius, and track width.

The command configuration uses a PathFollowerCommandGeneratorCfg with:

```
Python
        lookAheadDistance = 0.4
        maxSpeed = 1
        path_frame = "world"
        debug_vis = True
```

The environment runs with simulation time step 0.005 s, decimation 25, and episode length 60 s. Termination conditions include time-out, chassis contact, and goal reached within a 0.3 m threshold.

12. Quantitative Evaluation

To evaluate the effectiveness of the hybrid navigation system, we conducted scenario-based experiments comparing the baseline ViPlanner (vi) with the proposed hybrid approach (hyb) across multiple environments, including clear roads, regular obstacles, wide obstacles, narrow passages, and complex obstacle configurations. The evaluation metrics include collision rate, success rate, average time to reach the goal, switching latency, and minimum distance to obstacles.

	Collision Rate		**Success Rate**		**Avg time to goal**		**Switching latency**		**Min dist to obstacles**	
Scenario	vi	hyb	vi	hyb	vi	hyb	vi	hyb	vi	hyb
Clear road	0%	0%	100%	100%	36s	36.3s	N/A	N/A	N/A	N/A
Regular size obstacle	0%	0%	100%	100%	42.1s	41.9s	N/A	N/A	0.43m	0.45m
Wide obstacle	100%	0%	0%	80%	N/A	1m46s	N/A	0.2s	0	0.33m
Narrow Passage	60%	0%	40%	100%	2m06s	2m13s	N/A	0.29s	0	0.42m
Complex obstacles in narrow passage	100%	0%	0%	60%	N/A	2m36s	N/A	0.44s	0	0.3m

As shown in the quantitative results table, both approaches perform similarly in simple environments, achieving 0% collision rate and 100% success rate on clear roads and regular obstacle scenarios, with comparable time-to-goal performance. However, in more challenging environments, significant differences emerge. In the wide obstacle scenario, ViPlanner fails completely (100% collision rate, 0% success), while the hybrid system achieves 0% collision rate and 80% success by successfully switching to D* Lite with a switching latency of approximately 0.2s.

Similarly, in the narrow passage scenario, the hybrid system improves success rate from 40% to 100% while maintaining safe clearance from obstacles (0.42 m vs. 0 m in baseline). In the most complex scenario, involving multiple obstacles in a narrow passage, the hybrid system reduces collision rate from 100% to 0% and achieves a 60% success rate, demonstrating significantly improved robustness.

Overall, the results show that the hybrid system maintains comparable efficiency in simple environments while substantially improving safety, success rate, and obstacle clearance in challenging scenarios. The switching latency remains low (<0.5 s), indicating that the additional safety layer introduces minimal overhead while providing strong reliability benefits.

System status

Our project has successfully culminated in the development and validation of a Hybrid Neuro-Symbolic Navigation Framework deployed within the NVIDIA Isaac Sim environment. Moving beyond the conceptual phase, we have transitioned our primary testing platform to the Limo mobile robot, implementing a high-fidelity simulation that accurately mirrors real-world kinematics and sensor noise.

The current system state is summarized by the following architectural milestones:

Advanced Control and Motion Planning

We have optimized the low-level trajectory tracking by implementing a lookahead-based P controller. By introducing a steering deadzone and a dynamic lookahead distance, we have eliminated the micro-oscillations and "wobbling" typical of raw neural network waypoint tracking, achieving high-speed stability in narrow corridors.

Intelligent Arbitration Logic & Dynamic Reversion:

The navigation pipeline now features a robust Multi-Layer Bi-Directional Arbitration module. The system defaults to ViPlanner for its superior semantic intuition and smooth trajectory generation. To manage transitions, it employs a temporal fear buffer (fear_buffer) that monitors the network's internal uncertainty. If the uncertainty exceeds a threshold of 0.5 over consecutive frames, or if the target lies outside the current Field of View (FoV), the system executes an instantaneous fallback to D* Lite to ensure deterministic safety.

Critically, we have implemented a recovery protocol: once the D* Lite planner successfully navigates the robot away from the high-risk zone and the Combined Checker validates that the forward path is clear of semantic and physical obstacles, the system dynamically reverts control to ViPlanner. This ensures that the robot resumes its high-agility, neural-based navigation as soon as nominal conditions are restored, maximizing the efficiency and smoothness of the final path.

Deterministic Safety via Global Memory:

To resolve the "short-sightedness" of reactive models, we engineered a persistent world-frame obstacle memory. This module aggregates depth sensor data into a global occupancy grid, allowing the D* Lite planner to navigate complex traps and blocked scenarios using historical data that has long passed the camera's current frustum.

Performance Benchmarking:

The hybrid framework maintains an inference latency of <50ms. This was achieved through a custom linear interpolation and resampling pipeline that aligns the symbolic D* Lite paths with the tensor shapes expected by the physics engine, ensuring zero-overhead switching between planners.

Project logistics (project schedule personnel responsibilities)

Project Actual Timeline (Week 1 starts at Feb 9)

Week No.	**Phase**	**Description**
1	Env Setup	Installed and configured CARLA Simulator on a Windows-based WSL environment Verified core functionalities, including vehicle spawning, map switching, and basic control actions
2	Env Setup	Conducted an in-depth review of the ViPlanner paper and its GitHub repository Identified that the intended environment is based on NVIDIA Isaac Sim / Isaac Lab with CARLA map

		integration Attempted to transition to a WSL2-based Isaac Sim setup, but encountered compatibility issues due to non-native environment support
3	Env Setup & Infra Exploration	Explored cloud-based GPU solutions to support simulation Initially evaluated Lambda Labs, but found it cost-inefficient due to continuous billing (instances cannot be paused) Transitioned to RunPod and successfully provisioned an RTX 4090 instance Installed Isaac Sim and Isaac Lab along with all required dependencies Encountered limitations where available templates did not support graphical rendering, restricting simulations to headless execution Attempted to use NVIDIA Isaac Sim streaming, but this failed due to lack of UDP support in the RunPod environment
4	Env Stabilization & System Integration	Transitioned to a native Linux environment on a teammate's RTX 4090 machine to resolve prior limitations Configured remote access using AnyDesk and successfully established a stable development setup Explored Isaac Sim and successfully ran the ViPlanner demo using the ANYmal model Replaced the quadruped platform with a four-wheel vehicle by redesigning the actuation policy and control rules Conducted extensive scenario-based testing of ViPlanner Identified strengths (robust navigation in standard conditions) and limitations (e.g., sensitivity to limited field-of-view and complex obstacle layouts)
5, 6	D* Lite Implementation	Implemented the D* Lite algorithm for dynamic path planning Verified core functionality on grid-based maps, including path generation and replanning behavior

7	D* Lite Integration & Safety Mechanism	Integrated the D* Lite module into the existing planning codebase Enhanced the safety checker to evaluate ViPlanner's internal "fear" metric and trigger fallback when the trajectory is deemed unsafe Designed safety rules to ensure D* Lite generates and executes safe trajectories upon activation Conducted testing of the integrated system; however, D* Lite was unable to consistently generate correct or feasible trajectories in the current setup
8	D* Lite Debug	Added detailed logging to debug grid map generation, local goal selection, and decision-making processes Introduced obstacle clearance and inflation techniques to prevent trajectories from hugging walls and improve safety margins Identified critical reference frame mismatches between camera and world coordinates, as well as axis misalignment between the vehicle model and grid map Resolved these inconsistencies, enabling the vehicle to generate valid trajectories using D* Lite
9,10	Integration Test & Benchmark	Conducted system-level testing by designing diverse stress scenarios to evaluate planner robustness and safety Identified a latency issue where ViPlanner's reactive "fear" signal was triggered too late for effective decision-making Enhanced the safety checker to proactively evaluate proposed trajectories and trigger earlier intervention when unsafe behavior is detected Refined vehicle actuation logic to prevent abrupt control changes ("snapping") and improve overall stability and safety during navigation

Personnel responsibilities

Ting-You Chen	- Lower-Level Actuation

	- Viplanner Data Flow Analysis - D* Lite Algorithm - D* Lite Integration - System Troubleshoot & Safety Design - Fallback Algorithm - System test & benchmark
Guangzhou Cai	- Early WSL+Ubuntu setup - High-speed Computing Hardware purveyor - Isaac-Sim Limo diff-drive exploration - Carla-map exploration - Simulation vehicle testing - Viplanner_Demo Testing and Benchmarking
Ziang Jiao	- Cloud Infra Exploration - Isaac Sim env setup - Viplanner integration to 4 wheel robot - Test Scenario Setup - D* Lite Integration - System Troubleshoot & Safety Design - Fallback Algorithm - System test & benchmark

Challenges faced and solutions

The development process revealed several non-trivial technical hurdles, particularly regarding simulation fidelity and the integration of disparate navigation paradigms.

Vehicle Physics

Problem: Initial efforts to deploy the system in Isaac Sim encountered significant API compatibility issues regarding rigid-body instancing and actuator behavior of vehicle. Specifically, standard wheel-based models failed to respond to lower-level velocity commands due to mismatched physics scene configurations and read-only rigid body properties.

Solution: We performed a comprehensive overhaul of the robot's configuration files, custom-coding the two-wheel differential drive dynamics to ensure the Limo's simulated actuators behaved realistically. This required standardizing on the Limo platform to maintain a consistent Jacobian for the motion controller.

Multi-Frame Coordinate Transformation Discrepancy

Problem: A critical hurdle was the misalignment between the OpenCV coordinate convention (used by the depth camera) and the ROS/Isaac Sim world frames. This discrepancy caused the robot to misinterpret the ground plane as a vertical obstacle and set local goals in invalid spatial coordinates, leading to phantom collisions and navigation failure.

Solution: We implemented a consistent TF pipeline to correctly project depth data from the camera frame to the world frame and occupancy grid. Through logging and real-time visualization of occupancy grids and local goals, we identified two key issues: (1) a frame mismatch between camera coordinates (x-depth, y-lateral) and world/actuator coordinates (y-depth, x-lateral), and (2) an axis inconsistency where the vehicle's forward direction was defined as 0 in the robot frame but mapped to the grid center (index 40) in the occupancy grid. These mismatches caused incorrect goal projection and straight-line trajectories into obstacles. After aligning the coordinate frames and axis definitions, trajectory generation and navigation behavior became consistent and reliable.

Search Optimization and Computational Latency

Problem: As the ViPlanner model weights are fixed, the system occasionally suffered from "hallucinations" where it would attempt to pass through walls or ignore obstacles not represented in its training distribution. The internal "Fear Value" often failed to spike until the robot was too close to avoid a collision.

Solution: We introduced a Hybrid Combined Checker as an external supervisor that cross-references the neural network's proposed trajectory against an 80×80 local occupancy grid. To ensure safety, we apply obstacle inflation (0.2 m) and additional clearance checks, providing a double layer of safety constraints that account for the robot's footprint and safe margins. Unlike reactive approaches, the checker validates the entire ViPlanner trajectory at proposal time, enabling proactive (active) safety verification. If any part of the trajectory violates constraints, the system overrides the plan and falls back to the deterministic D* Lite planner.

Mitigating Neural Network "Hallucinations"

Problem: We encountered a significant trade-off between the search range and system lag. A range of 120 units caused severe frame drops via AnyDesk/Remote connection, while a 50-unit range was insufficient to find paths around large-scale blockages (e.g., 5.5m walls).

Solution: We determined that an 80x80 search grid provided the optimal balance. To further improve path realism, we transitioned from 4-connection to 8-connection grid logic, allowing the robot to plan diagonal trajectories that better represent the non-holonomic constraints of a car-like vehicle.

System improvements needed for the future

To further enhance the robustness and passenger experience of the Hybrid Neuro-Symbolic Navigation Framework, several key areas for future exploration have been identified:

Actuation Stability and Kinematic Refinement:

The Limo platform utilizes holonomic wheels, which presents a control challenge different from standard Ackermann steering. The current low-level logic exhibits instability and wobbling when a velocity difference occurs between the two wheels. Future work should focus on stabilizing the differential drive dynamics to ensure a smoother, more reliable response that accurately reflects real-world kinematics.

Path Smoothing for Passenger Comfort:

While transitioning from 4-connection to 8-connection grid logic improved path realism, the D* Lite algorithm still produces "zigzag" patterns. To align with real-world driving situations and provide a more comfortable experience for passengers, implementing smoothing techniques—such as Cubic Spline or Bezier curve interpolation—could eliminate these sharp, unnatural movements.

Hierarchical Navigation for Long-Range Scaling:

ViPlanner’s performance is primarily optimized for local navigation, and setting targets at significant distances can lead to performance degradation or local minima traps. A potential future improvement is a stage-based target system that breaks a distant global goal into manageable intermediate waypoints. This would allow the vehicle to "perceive and run" over longer distances while maintaining the high-agility performance of the neural planner.

Computational Optimization:

During the project, it was observed that the simulation environment did not fully utilize all available GPU resources. Future iterations could involve reconfiguring the hardware acceleration settings within Isaac Sim to ensure the vehicle runs more smoothly and with even lower inference latency.

Evaluation with Diverse Obstacles:

While the current testing focused on static wall layouts, the architecture is designed to handle Out of Distribution scenarios. Future experiments should include a wider variety of obstacles, such as

other cars or dynamic agents, to further validate the reliability of the Checker and switching logic in complex urban environments.